

Post-Lab Reflection

Task 1: Final Design Description

We didn't make any changes to our robot's design in the end. It's the same as what we planned in the pre-lab. Our original design was pretty solid and worked well enough to get the job done, so we didn't feel the need to move the sensors or change any parts.

Task 2: Performance Review and Shortcomings

The robot managed to complete all the tasks we planned for it, but its performance in the lab was very different from our tests at home, which was a surprise. We ran into a few main problems:

- **Drifting Problem:** The robot kept drifting to the left as it moved. This was a much bigger problem in the lab than at home. At home, it was only off by 1-2 cm, but in the lab, the error grew to 5 cm and sometimes even 10 cm.
- **Inaccurate Sensor Readings:** The distance sensor wasn't very precise. It was supposed to stop the robot 40 cm from the wall, but it usually stopped somewhere between 35 cm and 45 cm. We think the drifting made this problem worse. There was one really bad run where after hitting the wall, the robot was supposed to back up to 40 cm but only moved 5 cm. We think this happened because the robot drifted so far left that its sensor wasn't pointing at the board anymore, but was measuring the distance to the wall far behind it.
- **Lab vs. Home Environment:** The biggest surprise was how poorly the robot performed in the lab compared to at home. This made us realize our code wasn't prepared for different environments. The lab floor seemed to have a lot more friction than the wood floor at my house, which likely caused the distance errors.

On a positive note, the bump sensor worked perfectly every time without any issues.

Task 3: Future Improvements

We believe the robot's physical build is fine; the issues we found are mostly with the code. To make it perform better next time, we would focus on these software improvements:

- **Use PID Control to Drive Straight:** To fix the drifting, we should add a PID controller to our code. This would help us manage the two motors' speeds more precisely. By constantly checking how fast each wheel is spinning, PID can make real-time adjustments to keep the robot moving in a straight line.
- **Add a Friction Setting:** To handle different floor types, we could add a "friction" variable in the code. This way, we could adjust it based on the environment, like the grippy lab floor. This would help make our distance calculations more accurate and make the robot's performance more consistent no matter where we use it.